

Politechnika Warszawska

Wydział Matematyki
i Nauk Informacyjnych

Ensemble Learning – Research Project

Project 1: final plan

Non-linear and Interaction Features for Deep Forest

Authors:

Wojciech Jasiński

Sandra Adamiec

Warsaw, 20 October 2026

1 Introduction

Deep Forest [1] stacks layers of tree ensembles: each layer receives the original features and the predictions of the layer before it. These predictions are the only new columns a layer receives, and they carry limited information [2]. The trees also split on one column at a time, so an interaction with no marginal effect can be missed [3], for example a target that depends only on whether two features have the same sign.

The course topic suggests adding non-linearity or features that combine at least two features. We add both as extra columns given to the layers (Figure 1): transforms of a layer's predictions, non-monotone (their uncertainty and disagreement) or monotone (activations such as GELU), and pairwise products of two features or of a feature and a prediction, chosen by simple scores. Each variant will be compared with the *base cascade*, the same cascade without added columns.

Research hypotheses:

- **H1 Products help.** Adding feature–feature products chosen by correlation with the previous layer's error improves on the base cascade.
- **H2 Selection matters.** These products improve the cascade more than the same number of products drawn at random, and more than the same number chosen by correlation with the target.
- **H3 Prediction products help.** Adding feature–prediction products chosen by mutual information with the target improves on the base cascade.
- **H4 Non-monotone transforms help.** Adding the uncertainty and disagreement of the previous layer's predictions improves on the base cascade.
- **H5 Monotone activations help.** Adding monotone activations of the previous layer's predictions improves on the base cascade.

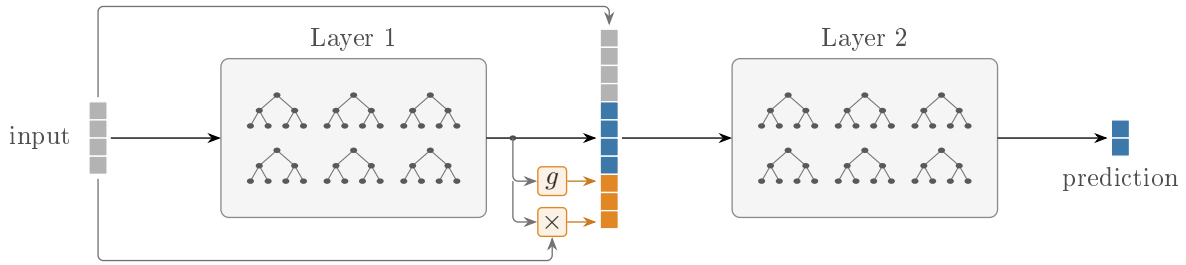


Figure 1: Deep Forest with the proposed change, shown for the first two layers. Besides input features (■) and predictions (■), a layer will receive new columns (■): transforms g of the predictions and pairwise products $\times$ of features with each other or with predictions. Feature–feature products also enter the first layer (not shown).

2 Related work

What layers pass on. Lyu et al. [4] prove that when layers pass on their predictions, the error of a two-layer forest can fall faster, as trees are added, than that of a single forest. A related analysis [5] finds that passing on interaction features instead of predictions gives the next layer more regions to work with and a lower risk of overfitting.

Interactions in forests. Random Intersection Trees [6] search for high-order interactions, and iterative random forests [7] use these trees to find stable interactions. hiDF [2] replaces the prediction features with such interactions. Interaction forests [8] split on two variables at once. MDGP-forest [9] builds new features between layers by genetic programming over sums, differences, products and ratios, and keeps those that are important alongside the layer's predictions.

Uncertainty as an input. In stacking, the entropy of each base model's class probabilities helped a linear combiner, but only when the probabilities themselves were kept [10]. For a deep regression cascade, Konstantinov [11] argued that raw prediction columns give trees no split on how far a layer's models agree. MVU-DF [12] also passes an uncertainty column to the next layer, but together with interaction features, resampling and sample weights, and does not test the column on its own.

Selecting from residuals. Reluctant Interaction Modeling [13] fits main effects first and then admits only interactions correlated with the residual. Our score (2) follows it. OpenFE [14] likewise scores candidate features by how much of the residual of out-of-fold predictions they explain.

Our position. Unlike hiDF, we keep the prediction features, so every variant is the base cascade plus some columns. To our knowledge, residual-selected products and uncertainty columns have not been added to Deep Forest on their own, nor tested against random and target-chosen products, monotone activations and a single forest.

3 Method

Cascade. Each layer will hold two random forests (RF) and two extremely randomized forests (ERF, with random split thresholds), with the defaults of the official Deep Forest package [15]. The package builds layer predictions from out-of-bag estimates. Because our variants choose columns using the labels, we will use three-fold cross-validation instead, as in the original paper [1], with the same folds at every layer and the columns chosen inside each fold. This gives out-of-fold (OOF) predictions P, the class probabilities of the four forests (in regression, their outputs). Each fold model of the next layer will be trained on $[X, P, \Phi]$, with the added columns Φ chosen on its own training rows. For test rows, P is the average of the three fold models' predictions, and each added column is the mean of its values computed from each fold model's predictions. These added columns, recomputed at every layer, are the only difference between variants (Table 1). We will implement the cascade in scikit-learn, reproducing the package's defaults, because the package has no option to add columns between layers.

Depth. Every variant will use the package's depth rule: layers are added until two in a row do not improve the OOF score on the training rows, up to 20, and the output is the average of the forests in the best layer. The score is balanced accuracy, our primary metric, instead of the package's accuracy; in regression it stays squared error. As depths may differ, we will also compare variants at each depth that all of them trained.

Products. Let x_i be a feature (standardized if numeric) and $p_{t,c}$ the mean prediction of the two forests of type $t \in \{\mathrm{RF}, \mathrm{ERF}\}$ for class c (in regression, one standardized output per type). The candidate columns are

$$\text{XX: } x_i\,x_j \;\; (i<j), \qquad\qquad \text{XP: } x_i\,p_{t,c}. \tag{1}$$

Each product variant will add its K best candidates, with $K = \min(d, \lfloor M_{XX}/4 \rfloor)$, where d is the number of input features after encoding and $M_{XX} = d(d-1)/2$ the number of XX candidates: no more products than features, and at most a quarter of the candidates, so that there is a real choice when features are few. K is our own choice; the sensitivity checks halve and double it.

Selection. Let y_c be the one-hot label (in regression, the target) and $\bar{p}_c$ the mean of the forests' OOF predictions on the original scale, so that the *layer error* is $r_c = y_c - \bar{p}_c$. An XX product ϕ will be scored by

$$s_{\text{target}}(\phi) = \max_c \left|\operatorname{corr}(\phi, y_c)\right| \qquad \text{or} \qquad s_{\text{residual}}(\phi) = \max_c \left|\operatorname{corr}(\phi, r_c)\right|. \tag{2}$$

Residual selection, adapted from Reluctant Interaction Modeling [13], uses the second score. In the first layer, which has no predictions, it equals target selection; target selection keeps the same products in every layer. XP products will be ranked by mutual information with y (scikit-learn's nearest-neighbour estimator, three neighbours). As it can favour products that mainly repeat the prediction, we will report how strongly the selected XP products correlate with P. Candidates constant on the training rows, such as products of two dummies of one categorical feature, are dropped before ranking.

Non-linear features. The package bins every column into quantiles before each layer, which we keep, and a split on a binned column uses only the order of its values. A monotone activation of a prediction, such as ReLU, sigmoid or GELU, therefore leaves its binned column unchanged; new split points need a transform that is non-monotone or combines several outputs. We propose three such columns $g(P)$, the uncertainty of each forest type t and the disagreement between the types:

$$H_t = -\sum\nolimits_c p_{t,c} \log p_{t,c}, \qquad D = \sum\nolimits_c \left|p_{\text{RF},c} - p_{\text{ERF},c}\right|. \tag{3}$$

In regression, H_t is the standard deviation of that type's tree predictions and D the absolute difference of the two means. The monotone variant will append an activation of every prediction column (regression outputs first scaled to $[0, 1]$, like probabilities). With binning, ReLU, sigmoid and GELU give the same binned columns, so the main runs will use the sigmoid and the check without binning will try all three. These columns may still help, by making the predictions more likely to be among the columns tried at a split and, without binning, by moving the ERF's random thresholds.

4 Datasets and preprocessing

We propose 16 classification and 16 regression datasets (Table 2): 31 from the UCI repository and California Housing from scikit-learn. Rows with a missing target will be dropped (Air Quality marks missing values with -200), and datasets larger than 20,000 rows will be subsampled once to 20,000, keeping related rows together and class proportions intact. Identifiers, timestamps, sampling weights and columns that reveal the target (such as call duration in Bank Marketing, casual and registered counts in Bike Sharing, motor UPDRS in Parkinsons, and the G1 and G2 grades in Student Performance) will be excluded. Missing values will be imputed with the median or mode, categorical features one-hot encoded and numeric features standardized, all fitted on training rows only.

Table 1: The compared methods. Feature–feature (XX) products enter every layer; the other added columns start at the second layer. Base cascade, non-linear, residual-XX and combined form a 2×2 ablation.

Method	Added columns	Chosen by	Purpose
Base cascade	none	–	reference
Random-XX	K XX products	random, redrawn at every layer	control (H2)
Target-XX	K XX products	correlation with target	control (H2)
Residual-XX	K XX products	correlation with layer error	proposed (H1, H2)
XP-MI	K XP products	mutual information with target	proposed (H3)
Non-linear	3 $g(P)$ columns	fixed, equation (3)	proposed (H4)
Monotone	sigmoid of predictions	fixed	proposed (H5)
Combined	$g(P)$ and residual-XX	as in their own rows	both changes together
RF	none	–	single-forest reference
RF + target-XX	K XX products	correlation with target	products, no cascade
LightGBM [16]	none	–	tuned reference

Table 2: Proposed datasets: rows × input features before our exclusions and encoding. Names are links.

Classification	Rows × feat.	Regression	Target	Rows × feat.
Sonar	208×60	Abalone	rings	$4{,}177 \times 8$
Spambase	$4{,}601 \times 57$	Auto MPG	mpg	398×7
QSAR Biodegradation	$1{,}055 \times 41$	Automobile	price	205×25
Steel Plates Faults	$1{,}941 \times 27$	Communities & Crime	ViolentCrimesPerPop	$1{,}994 \times 127$
Statlog Vehicle	846×18	Servo	`class` (continuous)	167×4
Statlog Heart	270×13	Energy Efficiency	Y1 (heating load)	768×8
Tic-Tac-Toe Endgame	958×9	Wine Quality (red)	quality	$1{,}599 \times 11$
Page Blocks	$5{,}473 \times 10$	Concrete Slump	compressive strength	103×7
German Credit	$1{,}000 \times 20$	Real Estate	price per unit area	414×6
Cardiotocography (NSP)	$2{,}126 \times 21$	Parkinsons Telemonitoring	total UPDRS	$5{,}875 \times 19$
Climate Crashes	540×18	Student Performance (maths)	G3	395×30
Phishing Websites	$11{,}055 \times 30$	Superconductivity	critical temperature	$21{,}263 \times 81$
SPECTF Heart	267×44	Bike Sharing (hourly)	count	$17{,}379 \times 13$
Default of Credit Card Clients	$30{,}000 \times 23$	Combined Cycle Power Plant	energy output	$9{,}568 \times 4$
Bank Marketing	$45{,}211 \times 16$	Air Quality	CO(GT)	$9{,}357 \times 14$
Adult	$48{,}842 \times 14$	California Housing	median house value	$20{,}640 \times 8$

5 Experimental protocol

Validation. We plan five outer folds repeated with three seeds: 15 runs per method and dataset, on the same splits. Related rows (identical inputs, same patient, material or day) will share a fold in every split, including the OOF ones, with classes stratified. Seeds will be fixed; splits, configurations, library versions and test predictions will be saved.

Hyperparameters and tuning. Cascade settings (Table 3) will be the same for every variant and are not tuned. Only the booster will be tuned: LightGBM [16], by 10 random settings from our ranges (learning rate 0.01–0.3, depth 3–8, row and column subsampling 0.6–1, L2 penalty 0.1–10) on three inner folds, with up to 1,000 trees, early stopping and the primary metric. Our other fixed choices are the 20,000-row cap and 400 trees for the single forests, as many as the four forests of one layer.

Sensitivity checks. Each value in the last column of Table 3 will be tried on its own, on all datasets with one seed (five runs), and reported without tests. The values of K will be tried for random-, target- and residual-XX, the other values for the base cascade and the residual-XX, non-linear and monotone variants.

Table 3: Cascade hyperparameters. m is the number of columns a layer receives; the last column gives the values tried in the sensitivity checks.

Hyperparameter	Value	Source	Also tried
Forests per layer	2 RF + 2 ERF	DF21 [15]	4 + 4 [1]
Trees per forest	100, fully grown	DF21	500 [1]
Features per split	$\sqrt{m}$	DF21	–
Column binning	255 quantile bins	DF21	none
Layer predictions	3-fold OOF	Zhou & Feng [1]	5 folds
Depth	until 2 layers bring no gain, ≤ 20	DF21	–
Depth score	balanced accuracy; squared error	ours	–
Products added, K	$\min(d, \lfloor M_{XX}/4 \rfloor)$	ours	$\lceil K/2 \rceil$ and $2K$

Compute. We timed single RF and ERF trees in scikit-learn on one core of an Apple M4, on synthetic data with 1,000 to 64,000 rows and 30 or 120 columns. Fit time grew about as $n^{1.0\text{–}1.2}\, m^{0.4\text{–}0.6}$ in rows n and columns m. Applied to every planned fit, with four to six trained layers per cascade and including LightGBM tuning and mutual-information scoring, this gives about 70–110 CPU-hours for the main experiment and 120–190 for the sensitivity checks (mostly the 500-tree check), so 190–300 in total, or one to two days on the laptop's ten cores.

Metrics and aggregation. Primary metrics will be balanced accuracy (classification) and RMSE (regression); log-loss, macro-F1, MAE and run time will be secondary. Per dataset we will report mean and standard deviation over the 15 runs; across datasets, mean ranks, the median change from the base cascade and win/loss counts, overall and by task.

Statistical tests. Each hypothesis will be decided by the paired comparison it states, over the 32 datasets; H2 needs both of its comparisons. Per dataset we will compute the relative error reduction $(\bar{e}_{\text{ref}} - \bar{e})/\max(\bar{e}_{\text{ref}}, \bar{e})$, zero if both errors are zero, where $\bar{e}$ is the error averaged over the 15 runs (one minus balanced accuracy, or RMSE) and ref is the method compared against, so positive values favour the variant. A two-sided Wilcoxon signed-rank test at $\alpha = 0.05$, with Holm's correction over the six comparisons [17], will decide each hypothesis, which is supported only if its median reduction is also positive. Results by task, the combined variant, and a Friedman test with its rank diagram over all eleven methods will be reported but will not decide any hypothesis.

6 AI assistance

Claude Code (Fable 5.1, Opus 5.5) was used to search for and summarise literature, to review the methodology, and to draft and edit this plan, including the related-work section and the figure. The authors chose the research questions within the course topic, made the design decisions and checked the text. External code and our changes will be credited in the report.

References

[1] Z.-H. Zhou and J. Feng. Deep Forest. *National Science Review*, 6(1):74–86, 2019.

[2] Y.-H. Chen, S.-H. Lyu, and Y. Jiang. Improving Deep Forest by Exploiting High-order Interactions. *IEEE ICDM*, 1030–1035, 2021.

[3] R. Blum, M. Hiabu, E. Mammen, and J. T. Meyer. Pure interaction effects unseen by Random Forests. *Comput. Stat. Data Anal.*, 212:108237, 2025.

[4] S.-H. Lyu, Y.-X. He, and Z.-H. Zhou. Depth is More Powerful than Width with Prediction Concatenation in Deep Forest. *NeurIPS*, 2022.

[5] S.-H. Lyu, Y.-H. Chen, and Z.-H. Zhou. A Region-based Analysis for the Feature Concatenation in Deep Forests. *Chinese Journal of Electronics*, 31(6):1072–1080, 2022.

[6] R. D. Shah and N. Meinshausen. Random Intersection Trees. *JMLR*, 15:629–654, 2014.

[7] S. Basu, K. Kumbier, J. B. Brown, and B. Yu. Iterative random forests to discover predictive and stable high-order interactions. *PNAS*, 115(8):1943–1948, 2018.

[8] R. Hornung and A.-L. Boulesteix. Interaction forests: Identifying and exploiting interpretable quantitative and qualitative interaction effects. *Comput. Stat. Data Anal.*, 171:107460, 2022.

[9] Z. Lin, Y. Xu, K. Liu, and L. Chen. MDGP-forest: A novel deep forest for multi-class imbalanced learning based on multi-class disassembly and feature construction enhanced by genetic programming. *Pattern Recognition*, 170:112070, 2026.

[10] B. Ženko and S. Džeroski. Stacking with an Extended Set of Meta-level Attributes and MLR. *ECML*, LNAI 2430, 493–504, 2002.

[11] A. V. Konstantinov. Deep Gradient Boosting for Regression Problems. *Computing, Telecommunications and Control*, 14(3):7–19, 2021.

[12] Q. Tan, Y. Wen, Y. Xu, K. Liu, S. He, and X. Bo. Multi-view uncertainty deep forest: An innovative deep forest equipped with uncertainty estimation for drug-induced liver injury prediction. *Information Sciences*, 667:120342, 2024.

[13] G. Yu, J. Bien, and R. Tibshirani. Reluctant Interaction Modeling. arXiv:1907.08414v3, 2023.

[14] T. Zhang, Z. Zhang, Z. Fan, H. Luo, F. Liu, Q. Liu, W. Cao, and J. Li. OpenFE: Automated Feature Generation with Expert-level Performance. *ICML*, PMLR 202:41880–41901, 2023.

[15] LAMDA-NJU. Deep Forest (DF21). Python package `deep-forest`, version 0.1.7.

[16] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *NeurIPS*, 2017.

[17] J. Demšar. Statistical Comparisons of Classifiers over Multiple Data Sets. *JMLR*, 7:1–30, 2006.